🍔 Restaurant Billing System

A modern, feature-rich restaurant billing and order management system built with React, Firebase, and Tailwind CSS. This application provides a complete solution for restaurant operations including menu management, order processing, payment handling, and customer management.

🌟 Features

🔐 Authentication System

- **Multi-Auth Support**: Email/password, Google OAuth
- **User Profiles**: Customer profiles with preferences and order history
- **Role-Based Access**: Customer, staff, admin, and owner roles
- **Profile Management**: User preferences, display names, and avatars

🍽️ Menu Management

- **Rich Menu Items**: 50+ items across multiple categories
- **Categories**: Main Course, Sides, Milkshakes, Desserts, Beverages
- **Item Details**: Prices, descriptions, nutritional information, preparation time
- **Search & Filter**: Real-time search and category-based filtering
- **Favorites System**: Users can mark and save favorite items

🛒 Shopping Cart & Orders

- **Dynamic Cart**: Add, remove, and modify quantities
- **Order Types**: Dine-in (with table selection), Takeaway, Delivery
- **Real-time Updates**: Cart persists across sessions
- **Order History**: Complete order tracking and history

💳 Payment Processing

- **Multiple Payment Methods**:
    - Cash payments
    - Credit/Debit cards

    - Digital wallets (UPI, PayPal)
    - Razorpay integration
- **Discount System**: Percentage and fixed amount discounts
- **Bill Splitting**: Split bills among multiple customers
- **Receipt Generation**: Digital receipts with order details

□ Delivery Management

- **Address Management**: Save and manage delivery addresses
- **Map Integration**: Interactive maps for address selection
- **Delivery Tracking**: Order status and delivery updates
- **Geolocation**: Automatic location detection

📊 **Customer Analytics**

- **Order History**: Complete transaction history
- **Total Spending**: Track customer lifetime value
- **Favorite Items**: Personalized recommendations
- **User Statistics**: Order frequency and preferences

🎨 **Modern UI/UX**

- **Responsive Design**: Mobile-first approach
- **Dark/Light Themes**: Customizable appearance
- **Smooth Animations**: Engaging user interactions
- **Accessibility**: ARIA labels and keyboard navigation
- **Progressive Web App**: Offline capabilities

🛠️ **Tech Stack**

Frontend

- **React 18**: Modern React with hooks and context
- **Vite**: Fast build tool and development server
- **Tailwind CSS**: Utility-first styling framework

- **Lucide React**: Beautiful icon library
- **React Router DOM**: Client-side routing

Backend Services

- **Firebase Auth**: Authentication and user management
- **Firestore**: NoSQL database for real-time data
- **Firebase Storage**: File and image storage
- **Cloud Functions**: Serverless backend logic

Development Tools

- **ESLint**: Code linting and formatting
- **PostCSS**: CSS processing
- **Autoprefixer**: CSS vendor prefixes
- **TypeScript Support**: Type-safe development

📁 Project Structure

```
src/
├── components/          # React components
│   ├── CartSidebar.jsx  # Shopping cart sidebar
│   ├── DeliveryAddressPage.jsx  # Address management
│   ├── LoginPage.jsx    # Authentication interface
│   ├── MenuPage.jsx     # Menu browsing and ordering
│   ├── OrderTypePage.jsx # Order type selection
│   ├── PaymentPage.jsx  # Payment processing
│   └── ReceiptPage.jsx  # Order confirmation
├── config/              # Configuration files
│   └── firebase.js      # Firebase configuration
├── contexts/            # React context providers
```

```
|   └── AppContext.jsx   # Global state management
├── data/               # Static data
|   └── menuData.js      # Menu items and categories
├── services/           # Business logic
|   ├── mapService.js    # Map and geolocation services
|   ├── paymentService.js # Payment processing
|   └── restaurantService.js # Core restaurant operations
└── styles/             # CSS and styling
```

🚀 Getting Started

Prerequisites

- Node.js 16.0 or higher
- npm or yarn package manager
- Firebase project setup

Installation

1. **Clone the repository**
2. git clone https://github.com/your-username/restaurant-billing-system.git
3. cd restaurant-billing-system
4. **Install dependencies**
5. npm install
6. **Firebase Configuration**
7. *# Create .env file in root directory*
8. VITE_FIREBASE_API_KEY=your_api_key
9. VITE_FIREBASE_AUTH_DOMAIN=your_auth_domain
10. VITE_FIREBASE_PROJECT_ID=your_project_id
11. VITE_FIREBASE_STORAGE_BUCKET=your_storage_bucket
12. VITE_FIREBASE_MESSAGING_SENDER_ID=your_sender_id

13. VITE_FIREBASE_APP_ID=your_app_id
14. **Start development server**
15. npm run dev
16. **Build for production**
17. npm run build

📱 Application Flow

Customer Journey

1. **Authentication**: Sign up/sign in using email or Google
2. **Order Type**: Choose dine-in, takeaway, or delivery
3. **Menu Browsing**: Search and filter menu items
4. **Cart Management**: Add items and manage quantities
5. **Address/Table**: Select delivery address or table number
6. **Payment**: Choose payment method and apply discounts
7. **Confirmation**: Receive order confirmation and receipt

Restaurant Staff Workflow

1. **Order Management**: View incoming orders
2. **Kitchen Display**: Track order preparation
3. **Payment Processing**: Handle cash and card payments
4. **Customer Service**: Manage customer inquiries
5. **Analytics**: View sales reports and trends

🔧 Configuration

Firebase Collections

```
COLLECTIONS = {
  CUSTOMERS: 'customers',
  ORDERS: 'orders',
  MENU_ITEMS: 'menuItems',
```

```
  CATEGORIES: 'categories',
  PAYMENTS: 'payments',
  ADDRESSES: 'addresses'
}
```

Environment Variables

```
VITE_FIREBASE_API_KEY=
VITE_FIREBASE_AUTH_DOMAIN=
VITE_FIREBASE_PROJECT_ID=
VITE_FIREBASE_STORAGE_BUCKET=
VITE_FIREBASE_MESSAGING_SENDER_ID=
VITE_FIREBASE_APP_ID=
VITE_RAZORPAY_KEY_ID=
VITE_GOOGLE_MAPS_API_KEY=
```

🎯 Key Features in Detail

Menu System

- **50+ Food Items**: Comprehensive menu with detailed descriptions
- **Nutritional Information**: Calories, protein, carbs, and fat content
- **Dynamic Pricing**: Flexible pricing with discount support
- **Category Management**: Organized menu categories
- **Search Functionality**: Real-time item search

Order Management

- **Multi-Type Orders**: Support for dine-in, takeaway, and delivery
- **Real-time Updates**: Live order status tracking
- **Order History**: Complete transaction records
- **Customer Preferences**: Saved favorites and preferences

Payment Integration

- **Razorpay Gateway**: Secure payment processing
- **Multiple Methods**: Cards, UPI, wallets, and cash
- **Automatic Calculations**: Tax, discounts, and tips
- **Receipt Generation**: PDF and email receipts

User Experience

- **Mobile Responsive**: Optimized for all devices
- **Fast Loading**: Optimized performance with Vite
- **Offline Support**: PWA capabilities for offline use
- **Accessibility**: WCAG 2.1 compliant design

📊 Database Schema

Customers Collection

```
{
  uid: string,
  email: string,
  displayName: string,
  phoneNumber: string,
  role: 'customer' | 'staff' | 'admin' | 'owner',
  createdAt: timestamp,
  lastLogin: timestamp,
  preferences: {
    notifications: boolean,
    language: string
  }
}
```

Orders Collection

```
{
```

```
  orderId: string,
  customerId: string,
  orderType: 'dine-in' | 'takeaway' | 'delivery',
  items: Array<{
    id: number,
    name: string,
    price: number,
    quantity: number
  }>,
  totalAmount: number,
  paymentMethod: string,
  status: 'pending' | 'confirmed' | 'preparing' | 'ready' | 'delivered',
  createdAt: timestamp,
  deliveryAddress?: string,
  tableNumber?: number
}
```

💳 Payment Integration

Razorpay Test Configuration

- **Test API Key**: rzp_test_1DP5mmOlF5G5ag
- **Mode**: Test mode (no real money charged)

Test Payment Details:

- **Card Number**: 4111111111111111 (Visa)
- **Expiry**: Any future date (e.g., 12/25)
- **CVV**: Any 3 digits (e.g., 123)
- **UPI**: success@razorpay

Payment Methods Supported:

1. **UPI Payment** - PhonePe, Google Pay, etc.
2. **Card Payment** - Credit/Debit cards
3. **Cash Payment** - Pay at counter/delivery

□ Security Features

- **Firebase Security Rules**: Secure database access
- **User Authentication**: Verified user accounts
- **Data Validation**: Input sanitization and validation
- **HTTPS Encryption**: Secure data transmission
- **Role-Based Access**: Permission-based feature access

🚀 **Performance Optimizations**

- **Code Splitting**: Dynamic imports for better performance
- **Image Optimization**: Optimized images and lazy loading
- **Caching Strategy**: Local storage for cart and preferences
- **Bundle Optimization**: Tree shaking and minification
- **CDN Integration**: Firebase hosting with global CDN

🧪 **Testing**

 # Run linting

npm run lint

 # Run type checking

npm run type-check

 # Run tests

npm run test

 # Run e2e tests

npm run test:e2e

📈 Future Enhancements

- [ ] **Kitchen Display System**: Real-time order management for kitchen
- [ ] **Inventory Management**: Stock tracking and low-stock alerts
- [ ] **Loyalty Program**: Points and rewards system
- [ ] **Multi-Restaurant Support**: Chain restaurant management
- [ ] **Advanced Analytics**: Business intelligence dashboard
- [ ] **Voice Ordering**: AI-powered voice commands
- [ ] **QR Code Menus**: Contactless menu access
- [ ] **Integration APIs**: Third-party delivery platforms

🤝 Contributing

1. Fork the repository
2. Create your feature branch (git checkout -b feature/AmazingFeature)
3. Commit your changes (git commit -m 'Add some AmazingFeature')
4. Push to the branch (git push origin feature/AmazingFeature)
5. Open a Pull Request

📄 License

This project is licensed under the MIT License - see the LICENSE file for details.

🙏 Acknowledgments

- **Firebase**: Backend infrastructure and real-time database
- **Tailwind CSS**: Utility-first CSS framework
- **Lucide Icons**: Beautiful icon library
- **React Team**: Amazing frontend framework
- **Vite**: Lightning-fast build tool

📞 Support

For support, email support@restaurantbilling.com or join our Slack channel.

🌟 Show Your Support

Give a ⭐ if this project helped you!

Built with ❤️ for the Restaurant Industry

Making restaurant operations seamless, one order at a time. { id: 1, name: "Burger", price: 199, quantity: 2 }], orderType: "delivery", // "dine-in", "take-away", "delivery" deliveryAddress: "123 Main St...", paymentMethod: "upi", // "card", "upi", "cash" subtotal: 398, discount: 50, total: 348, splitCount: 1, tableNumber: null, status: "paid", // "pending", "confirmed", "preparing", "ready", "delivered" createdAt: "2025-09-21T...", timestamp: 1726934567890 }

`payments` Collection:

```javascript
{
  orderId: "MMA1234567890",
  firebaseOrderId: "firebase_doc_id",
  paymentMethod: "upi",
  amount: 348,
  status: "completed", // "pending", "completed", "failed"
  paymentId: "razorpay_payment_id",
  razorpayOrderId: "razorpay_order_id",
  signature: "razorpay_signature",
  createdAt: "2025-09-21T...",
  timestamp: 1726934567890
}
```

users Collection:

```
{
  userId: "firebase_user_uid",
```

email: "user@example.com",

displayName: "John Doe",

phoneNumber: "+1234567890",

preferences: {

favoriteItems: [],

defaultAddress: "..."

},

updatedAt: "2025-09-21T..."

}

analytics Collection:

{

type: "order_placed",

orderId: "MMA1234567890",

amount: 348,

paymentMethod: "upi",

orderType: "delivery",

itemCount: 3,

createdAt: "2025-09-21T...",

timestamp: 1726934567890

}

🚀 **Key Features**

✅ **What's Working:**

- ✅ User authentication (Firebase Auth)
- ✅ State-based routing between pages
- ✅ Shopping cart functionality with localStorage

- ✅ Menu browsing with category filtering
- ✅ Payment processing (Razorpay integration)
- ✅ Order data persistence to Firebase Firestore
- ✅ Payment tracking and verification
- ✅ Receipt generation and printing
- ✅ Delivery address with map integration
- ✅ Order type selection (dine-in, takeaway, delivery)
- ✅ Discount and bill splitting features

🔧 **Recent Fixes:**

1. **Fixed JSX syntax errors** - Renamed AppContext.js to AppContext.jsx
2. **Added Razorpay test integration** - Working payment processing
3. **Implemented Firebase persistence** - Orders and payments now save to database
4. **Enhanced payment flow** - Proper error handling and user feedback

🧪 **Testing the Application**

Test a Complete Order Flow:

1. **Login**: Use Firebase auth (create account or use existing)
2. **Select Order Type**: Choose dine-in, takeaway, or delivery
3. **Browse Menu**: Add items to cart
4. **Payment**:
    - Select UPI/Card payment
    - Use test card: 4111111111111111
    - Check browser console for Firebase save confirmations
5. **Receipt**: View final order details

Firebase Database Verification:

- Open Firebase Console → Firestore Database

- Check orders and payments collections
- Verify data is being saved correctly

🧱 Tech Stack

- **Frontend**: React, Tailwind CSS
- **Backend**: Firebase (Firestore, Auth)
- **Payments**: Razorpay
- **State Management**: React Context + useReducer
- **Maps**: Custom map service integration
- **Build Tool**: Vite

🔐 Environment Setup

Required API Keys:

1. **Firebase** ✅ (Already configured)
2. **Razorpay** ✅ (Test key configured)
3. **Maps API** (Optional - for delivery address)

Development Commands:

```
npm run dev    # Start development server
npm run build  # Build for production
npm run preview # Preview production build
```

📞 Support & Contact

- Payments are in TEST mode - no real money charged
- All orders save to Firebase Firestore
- Check browser console for detailed logs
- Firebase project: hiresense-a7146